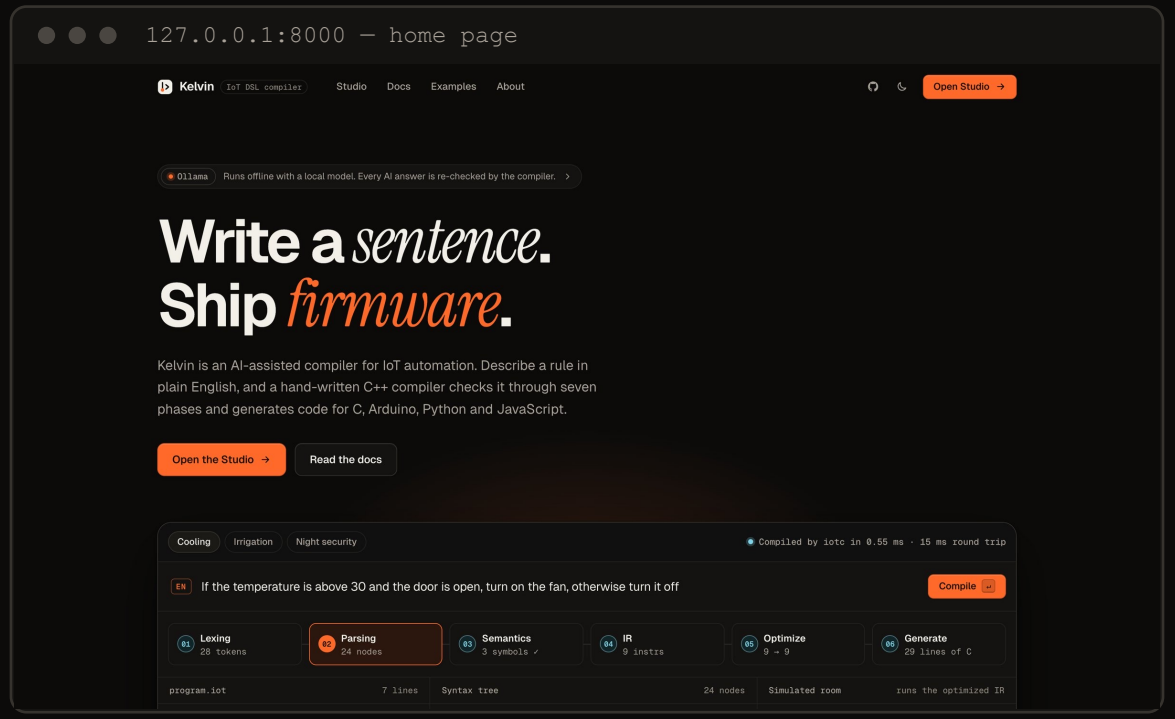
Kelvin

COMPILER DESIGN PROJECT

Write a *sentence*. Ship *firmware*.

An AI-assisted DSL compiler for IoT automation

Plain English in. A hand-written C++ compiler checks it in seven phases and generates C, Arduino, Python and JavaScript.



TEAM

R. Sanjay · Settipalli Tejovardhan · Ashutosh

Smart homes run on rules, but *people* don't write code

- **Automation is a set of rules:** if a sensor reads something, switch a device.
- **The people who want the rules** (home owners, farmers, facility staff) are not programmers.
- **AI understands English,** but it is sometimes wrong, and a wrong rule can switch off a heater or flood a garden.
- **Our answer:** let the AI draft the rule, and let a real compiler check every rule before it runs.

ENGLISH

"If the temperature is above 30 and the door is open, turn on the fan."

DSL · checked by iotc

```
SENSOR temperature : NUMBER
SENSOR door : STATE
DEVICE fan

IF temperature > 30 AND door == OPEN
THEN turn_on(fan)
```

Three parts, *one* pipeline

01

AI assistant

English or voice to DSL with free models: Ollama (local), Groq or Gemini, plus an offline converter.

Every answer is compiled before you see it.

02

Hand-written compiler

C++17, no Lex or Yacc. Lexer, LL(1) parser, AST, semantic and rule analysis, IR, CFG optimizer, code generation.

Output: C, Arduino, Python, JavaScript.

03

Website and Studio

Watch every phase, run the generated code, and simulate it in a smart-home digital twin.

Phone remote, share links, PDF report.

7
compiler phases

4
code targets

121
automated tests passing

4,865
lines of hand-written C++

From a sentence to *running code*

INPUT	FRONT END	MIDDLE	BACK END
English typed or spoken	**Lexer** lexer.cpp	**Three-address IR** ir.cpp	**Code generation** codegen.cpp: C · Arduino · Py · JS
AI assistant ai.py	**Parser → AST** parser.cpp · ast.cpp	**CFG & dataflow** cfg.cpp	**Runtimes** runtime/iot_runtime.*
DSL source by hand or by AI	**Semantic & rules** semantic.cpp · rule_analysis.cpp	**Optimizer, 8 passes** optimizer.cpp	**Digital twin** runs the optimized IR

No parser generators · no frameworks · Python standard library only · works offline

A small language with a *precise* grammar

smart_home.iot

```
# Smart home rules
SENSOR temperature : NUMBER
SENSOR door        : STATE
SENSOR motion      : STATE
DEVICE fan, light, ac

IF temperature > 30 AND door == OPEN
THEN turn_on(fan)
ELSE IF temperature > 26
THEN set_level(fan, 50)
ELSE turn_off(fan)

IF motion == ON OR temperature >= 35
THEN turn_on(light)
     set_level(ac, 22.5)
```

- **Declarations:** SENSOR name : NUMBER | STATE, and DEVICE names.
- **Rules:** IF … THEN … with any number of ELSE IF and an optional ELSE.
- **Conditions:** comparisons, AND, OR, NOT, parentheses and arithmetic (+ − × ÷).
- **Actions:** turn_on, turn_off, toggle, set_level.
- **Types:** NUMBER, STATE (ON, OFF, OPEN, CLOSED, TRUE, FALSE) and BOOL.

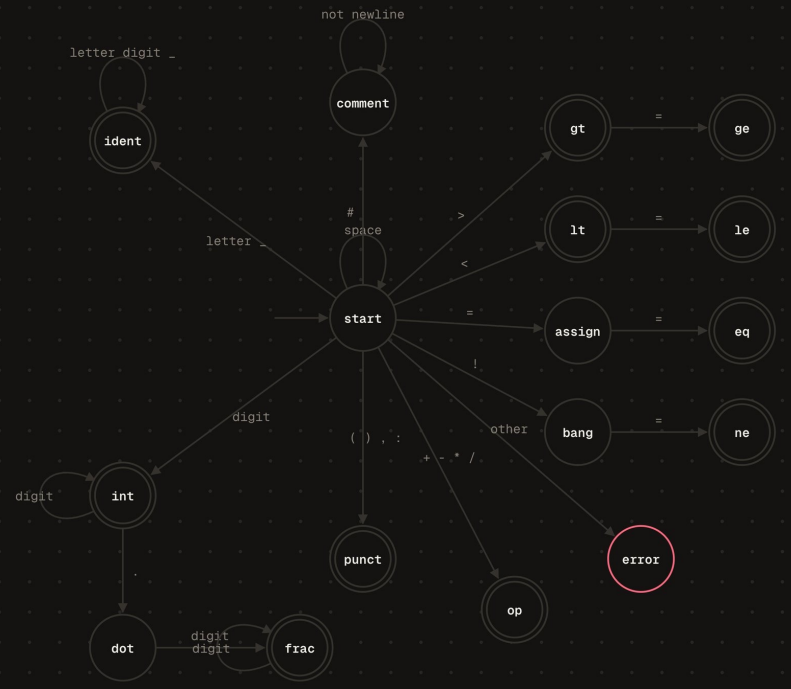
LL(1) 61 productions · 0 conflicts

The compiler computes FIRST, FOLLOW and the parse table from its own grammar.

Lexer and parser, written *by hand*

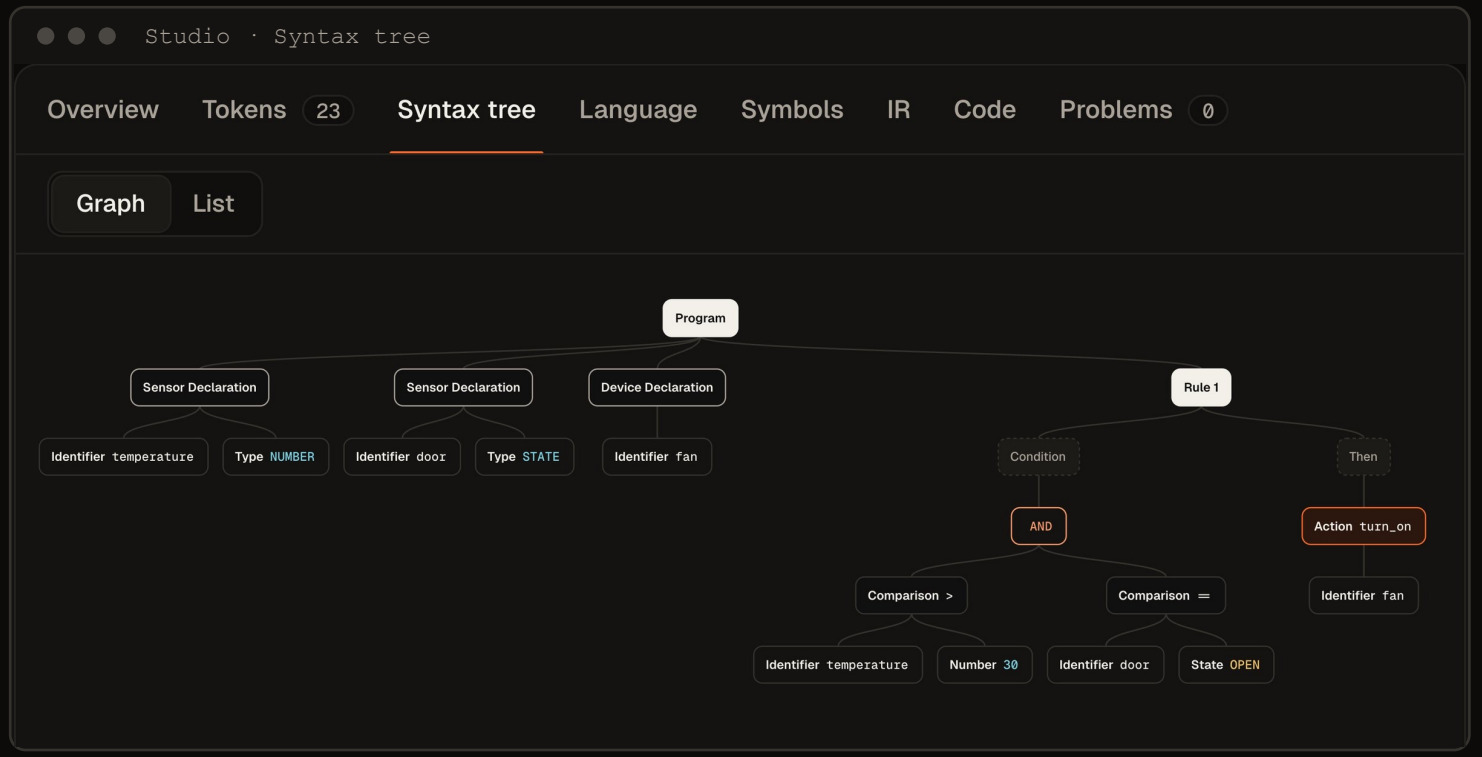
Scanner DFA — from the compiler

FIG. 1 **The scanner's automaton** 17 states, from the hand-written lexer



- **Lexer:** a hand-coded DFA with 17 states. It reads one character at a time and takes the longest match (maximal munch).
- **Errors with fixes:** a bad character or '=' instead of '==' is reported with its line, column and a one-click fix.
- **Parser:** recursive descent; each grammar rule is one C++ function (parseExpr, parseAnd, parseRelational …).
- **Error recovery:** panic mode, so several syntax errors are reported in one run.

The abstract *syntax tree*



- **One expression tree** for conditions and arithmetic: binary, unary and operand nodes.
- **Precedence is built into the grammar:** NOT binds tighter than AND, and AND tighter than OR.
- **Every node keeps its source position,** so hovering a node highlights the matching code, tokens, IR and C lines.
- **Drawn live:** pan, zoom and collapse, or switch to a list view.

Semantic and *rule* analysis

Symbol table and warnings — example 08

FIG. 4 **Symbol table and rule analysis** example 08_rule_analysis

NAME	KIND	TYPE	DECLARED	USES
temperature	sensor	NUMBER	line 2	8
door	sensor	STATE	line 3	1
fan	device	-	line 4	2
alarm	device	-	line 4	4
light	device	-	line 4	1

warning line 7: this condition can never be true, so its actions never run

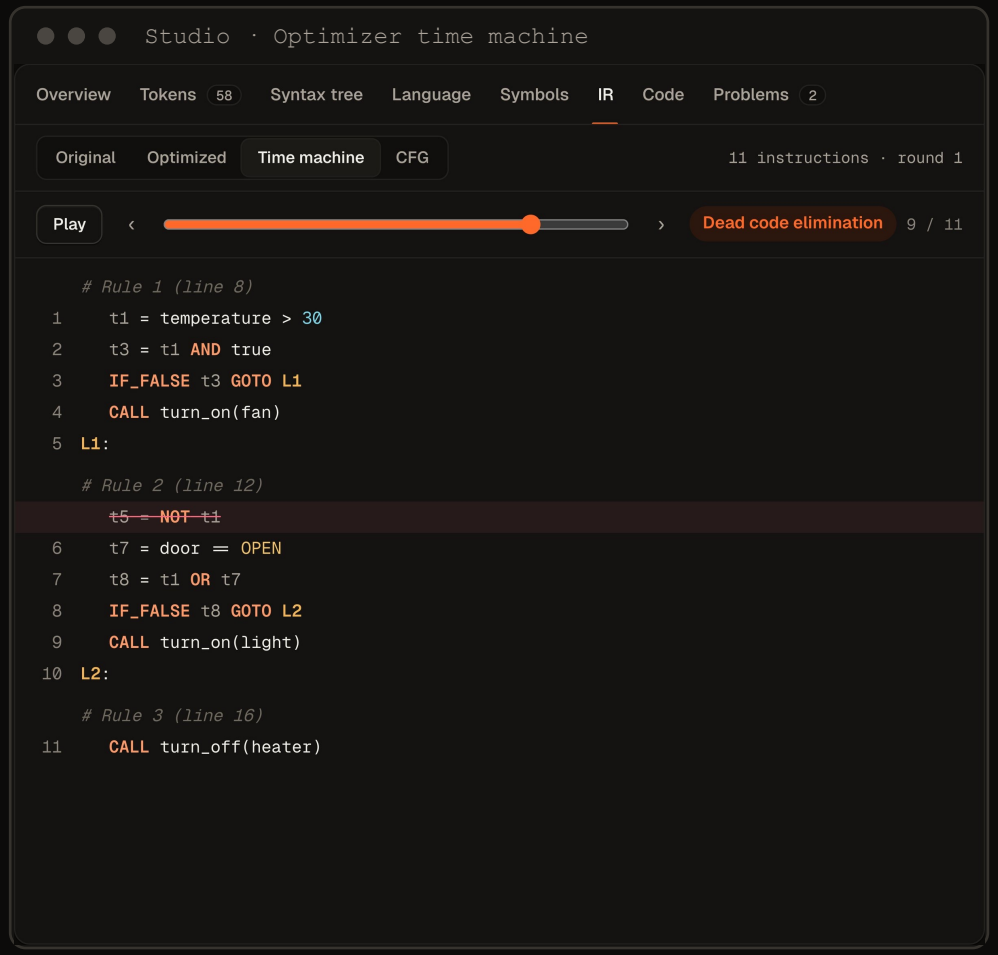
warning line 11: this condition is always true, so the rule always runs

warning line 16: this ELSE IF can never run: whenever its condition is true, an earlier condition of the same rule is already true

warning line 21: conflict: rule 4 turns 'fan' off while rule 3 (line 15) turns 'fan' on, e.g. when temperature = 31 and door = OPEN. The later rule wins

- **Symbol table and types:** every name declared once, NUMBER vs STATE checked, action arguments checked.
- **Helpful errors:** “did you mean 'temperature'?” and one-click fixes such as declaring a missing device.
- **Rule analysis:** conditions are put in normal form (DNF) and checked with interval and state-set reasoning.
- **It finds** impossible conditions, always-true conditions, unreachable ELSE IFs, and two rules fighting over a device, with example inputs.

Intermediate code and *optimization*



- **Three-address code** with temporaries (t1, t2) and labels (L1, L2); each instruction keeps its source span.
- **Basic blocks and a CFG;** liveness and available-expressions dataflow analysis.
- **Eight passes until nothing changes:** constant folding, algebraic simplification, CSE, copy propagation, constant branches, unreachable code, jumps, dead code.

21 → 10

instructions in example 07

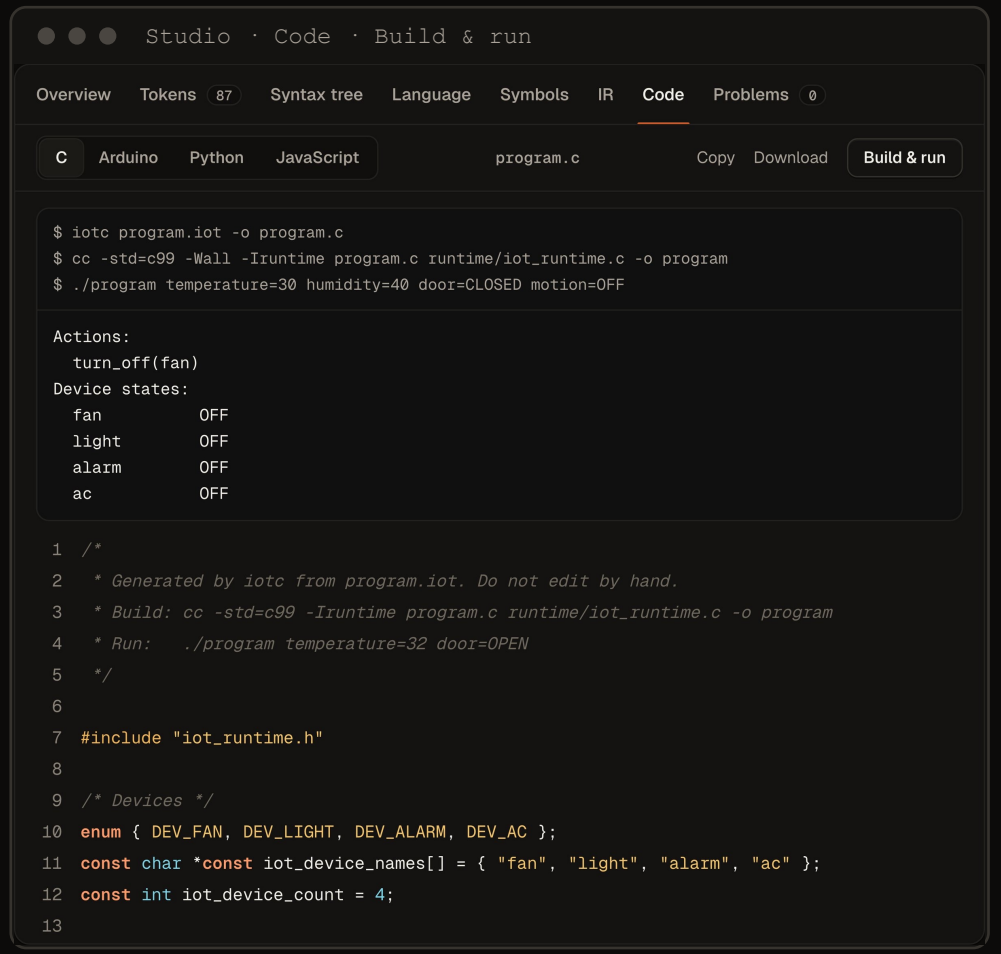
−52%

smaller after optimization

One compiler, *four* targets

C | Arduino | Python | JavaScript

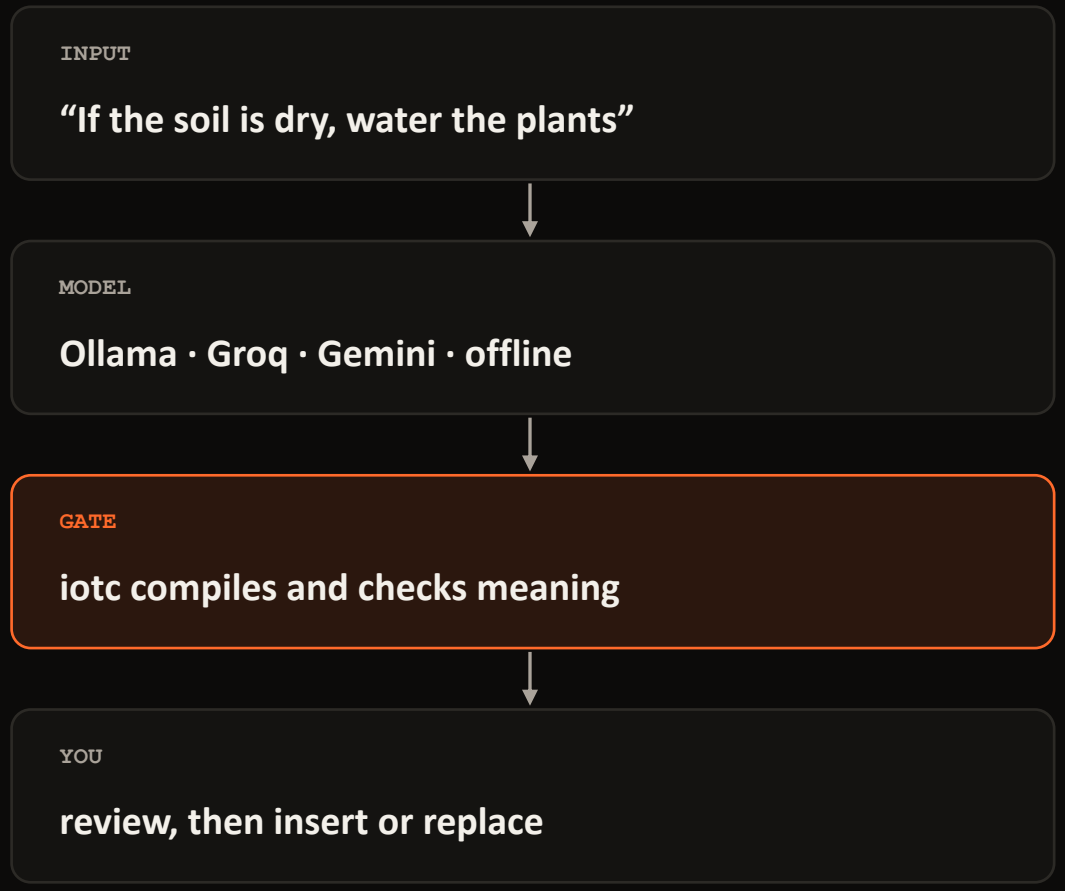
- **One structured emitter,** four back ends. If/else structure is recovered from the optimized IR, so the output reads like hand-written code.
- **C** links with our runtime; Build & run in the Studio compiles it with a real C compiler and runs it.
- **Arduino / ESP32** sketch with setup() and loop() and a pin table.
- **Tested:** for every example and several inputs, C, Python and JavaScript produce identical output.



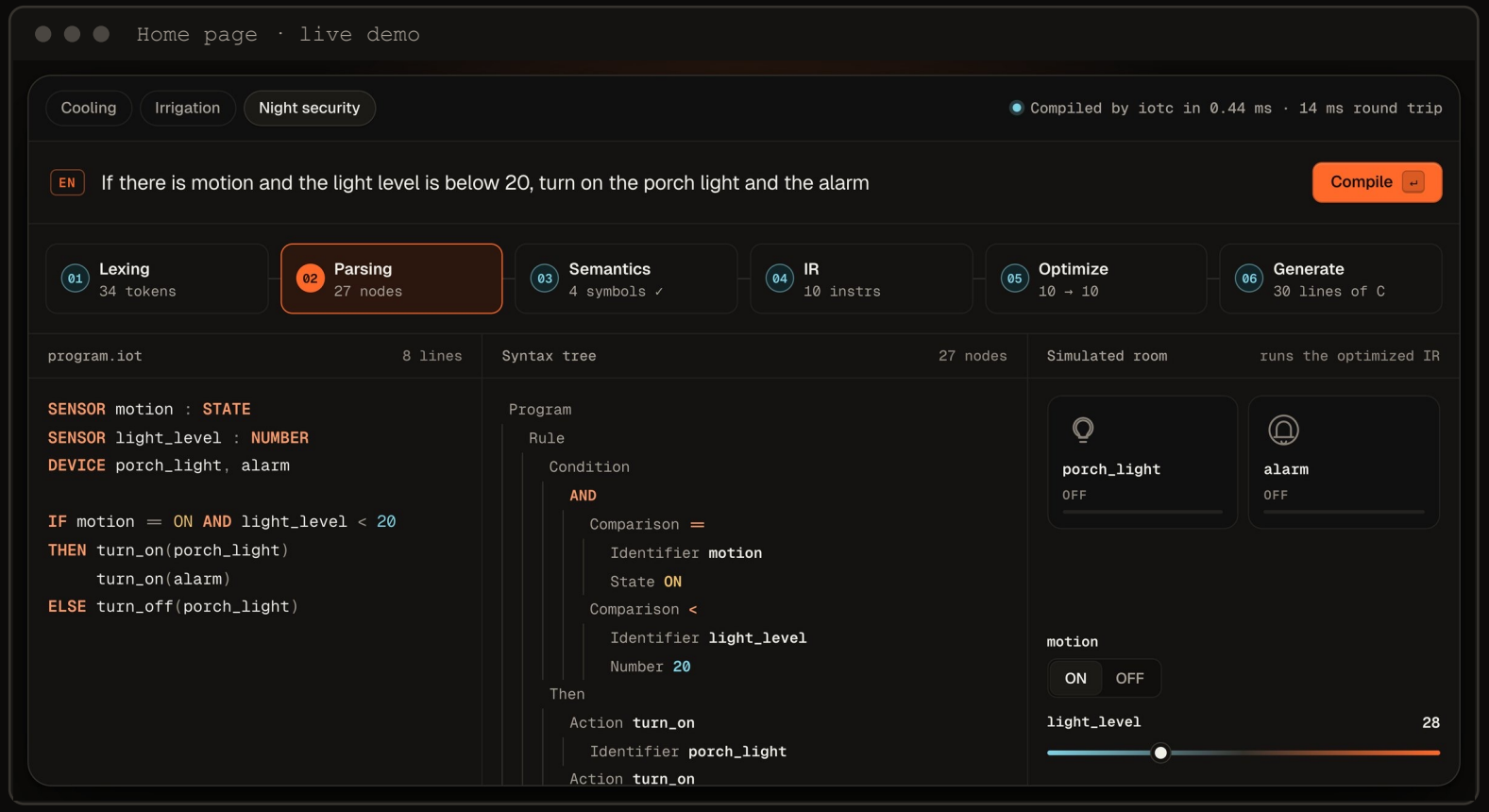
The model *proposes*. The compiler *decides*.

- **Free models:** a local Ollama model (qwen3), Groq or Gemini; an offline converter works with no AI at all.
- **Every answer is compiled.** If it has errors, the compiler's messages go back to the model for one repair attempt.
- **Meaning check:** every number in the sentence must appear in the program, and every sensor must be used.
- **AI fix** is offered only if the error disappears and no new error appears.



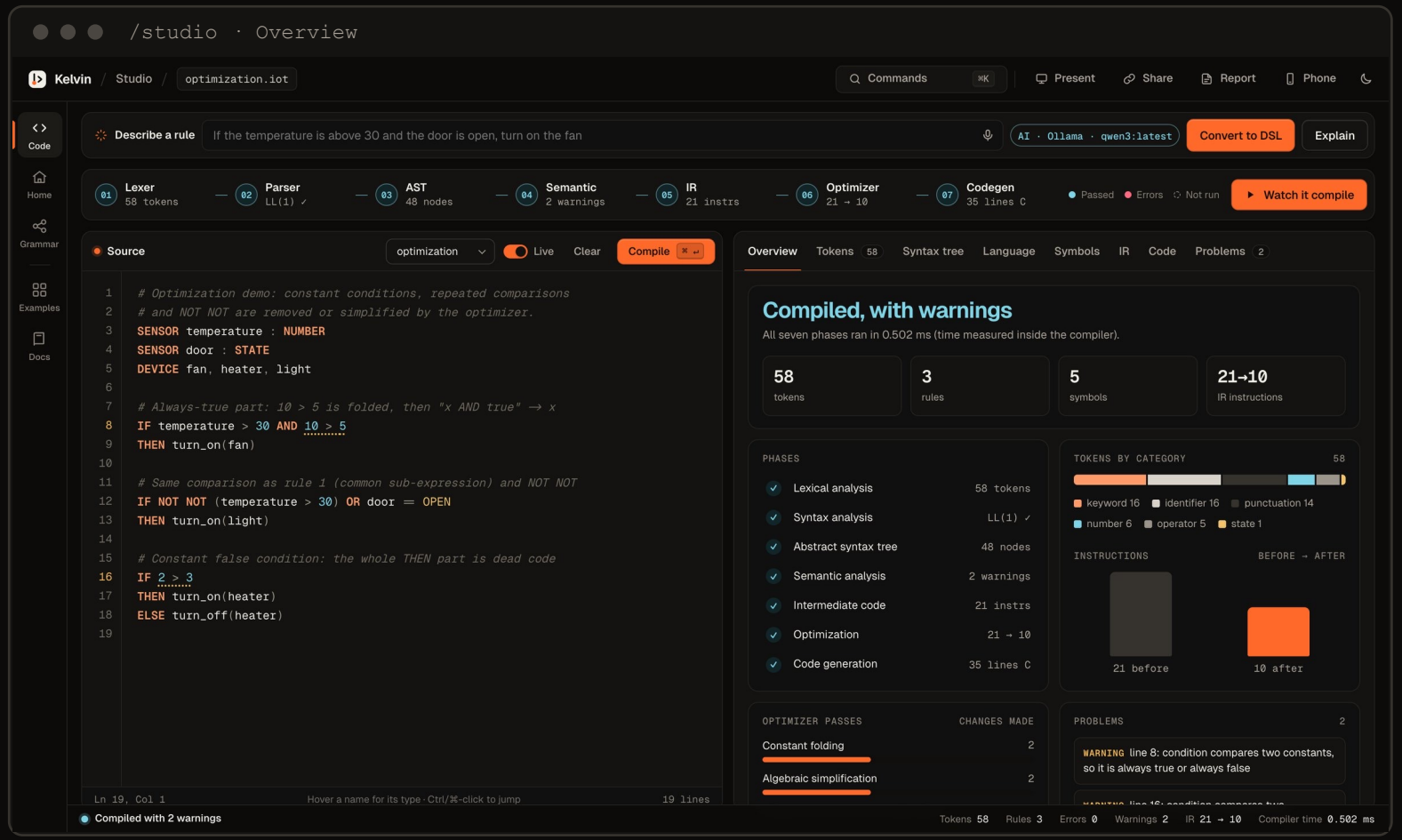


A home page with a *live* compiler demo



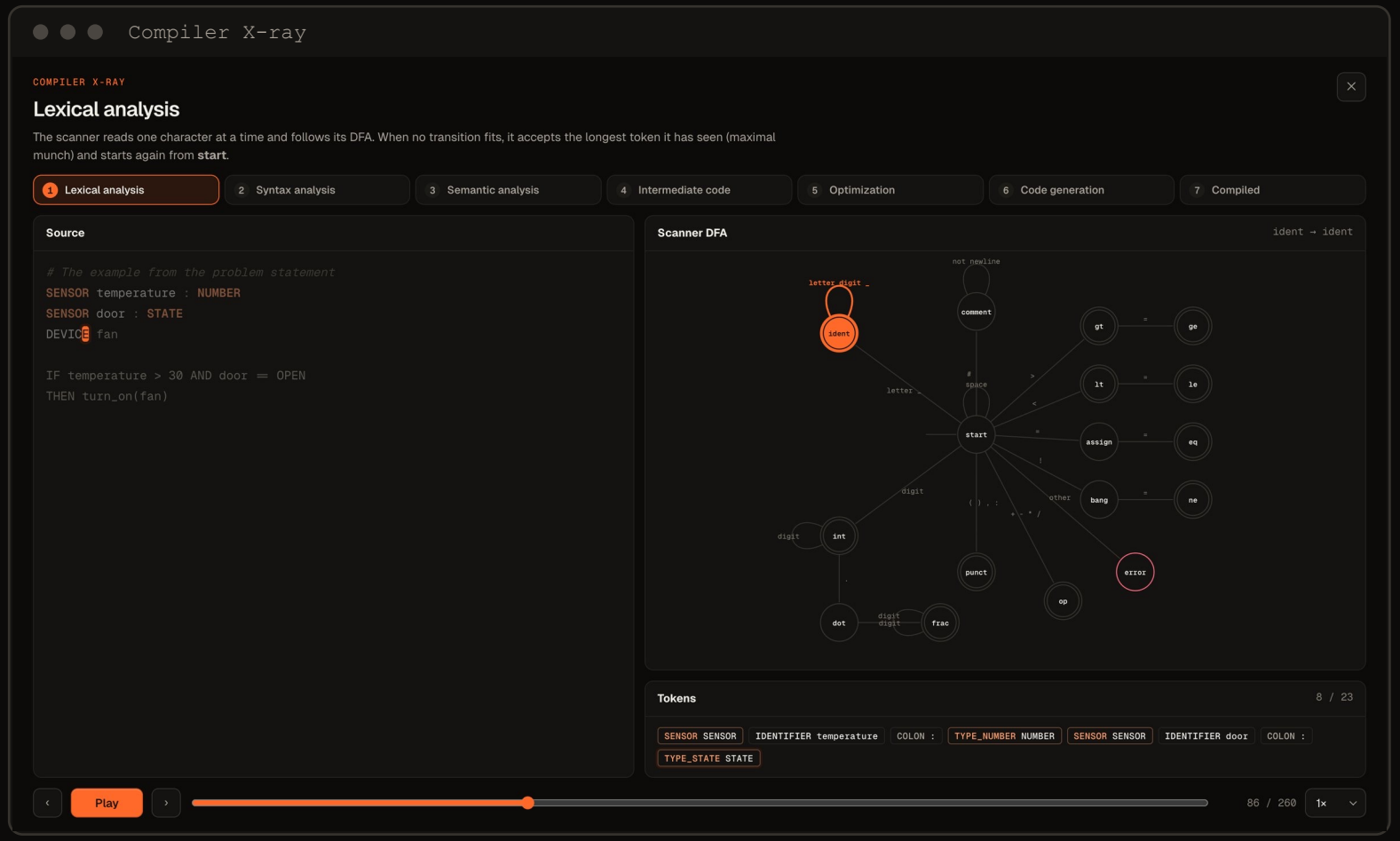
- **Not a recording:** each preset, or a sentence you type, is compiled by the real compiler.
- **Six phase boxes** show real results: tokens, tree nodes, symbols, instructions.
- **A simulated room** runs the optimized code; moving the slider switches the device.
- **Every number on the site** comes from the project (tests, code size, examples).

A workbench for *every* phase



- **Smart editor:** compiles as you type, autocomplete, types on hover.
- **Phase strip:** each phase shows what it produced; click to open it.
- **Tabs:** overview, tokens, tree, grammar, symbols, IR, code, problems.
- **Presentation mode,** command menu (Ctrl/⌘ K), share link and PDF report.

Watch the compiler *work*



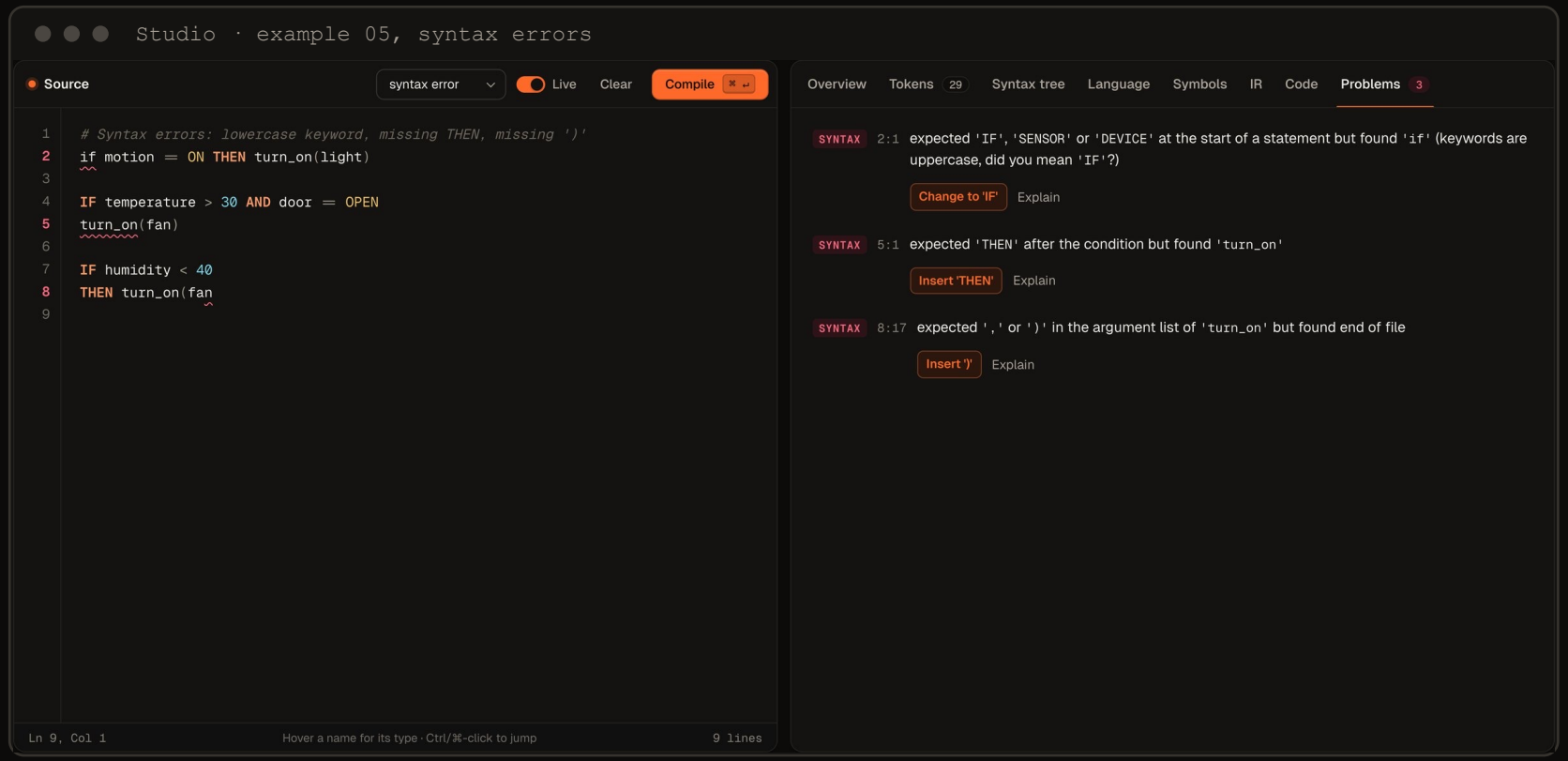
- **Replays a real compilation** from traces the C++ compiler records.
- **Scanner:** the DFA lights up character by character.
- **Parser:** the call stack grows and the tree builds node by node.
- **Then** symbol table, IR, each optimizer pass and the generated code.

The program runs a *house*



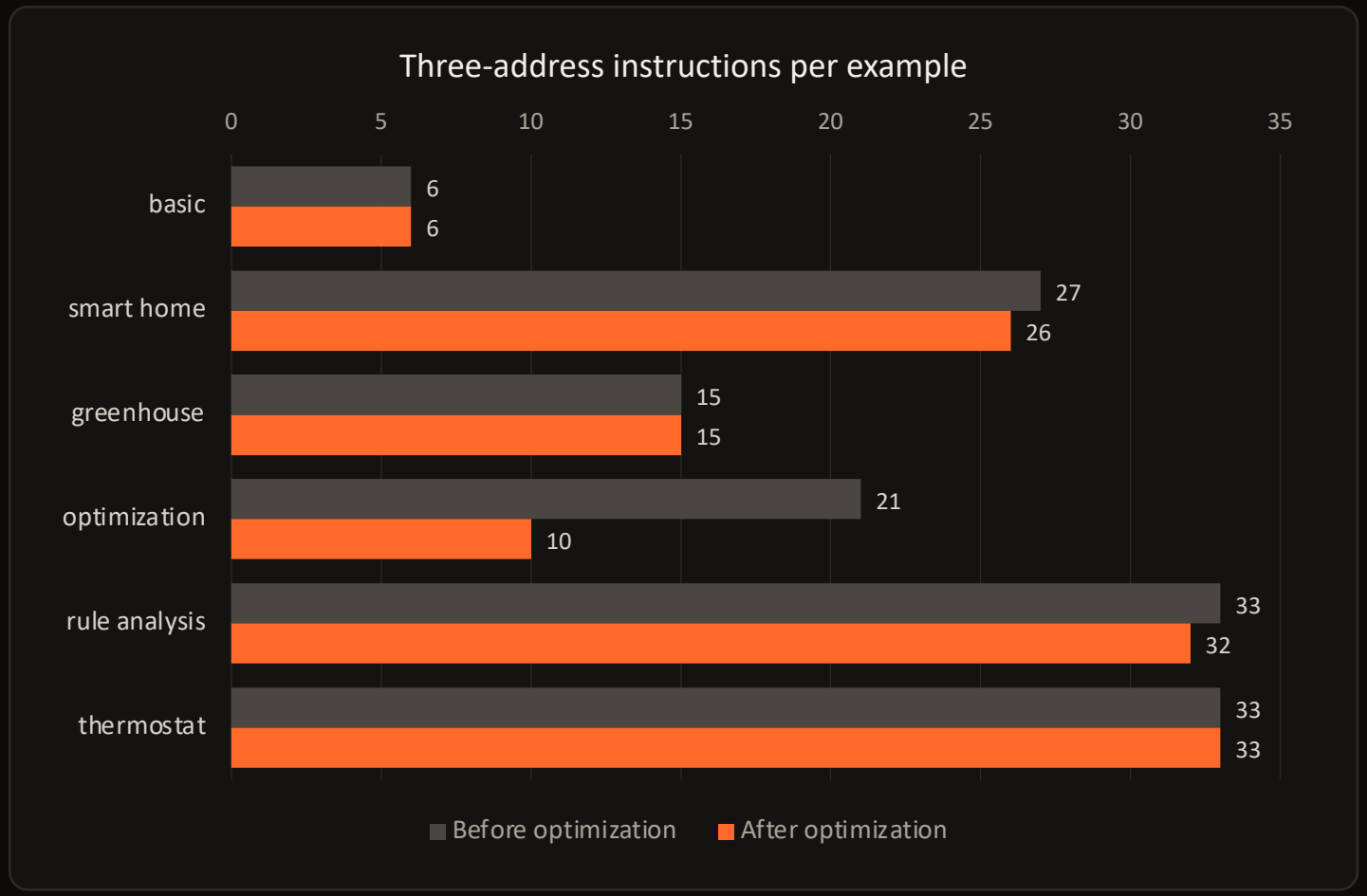
- **An animated floor plan** driven by the optimized IR in a small virtual machine.
- **Play a day:** sensors follow daily curves; heaters warm the room and pumps water the soil.
- **Conflicts show in red** when two rules fight over a device.
- **Phone remote:** scan a QR code and move the sensors from a phone.

Clear errors with *one-click* fixes



- **Every phase reports** its own errors with line and column.
- **Quick fixes** come from the compiler: “Change to 'IF'”, “Insert 'THEN'”, “Insert ')'”.
- **Failed phases are red;** later phases show “not run”.
- **Undo** works after every fix.

Measured, *not claimed*



102

compiler checks passing (tests/run_tests.sh)

19

Python unit tests: server, phone remote, AI checks

- C, Python and JavaScript give identical output for every example.
- Every Arduino sketch compiles.
- Grammar: 0 LL(1) conflicts.

Simple tools, *no* dependencies

Compiler	C++17, hand-written (no Lex/Yacc)
Server	Python 3, standard library only
Website	HTML, CSS, JavaScript; fonts bundled
AI	Ollama (local) · Groq · Gemini · offline
Tests	Shell script + Python unittest

terminal

```
$ make    # build the compiler
$ make run    # start the website
$ make remote    # …with the phone remote
$ make test    # run every test
```

Then open http://127.0.0.1:8000

Everything runs on one laptop, offline.

THANK YOU

From *intent* to firmware, checked at every step.

RS R. Sanjay

ST Settipalli Tejovardhan

A Ashutosh

Questions?

